The Orbis Journal

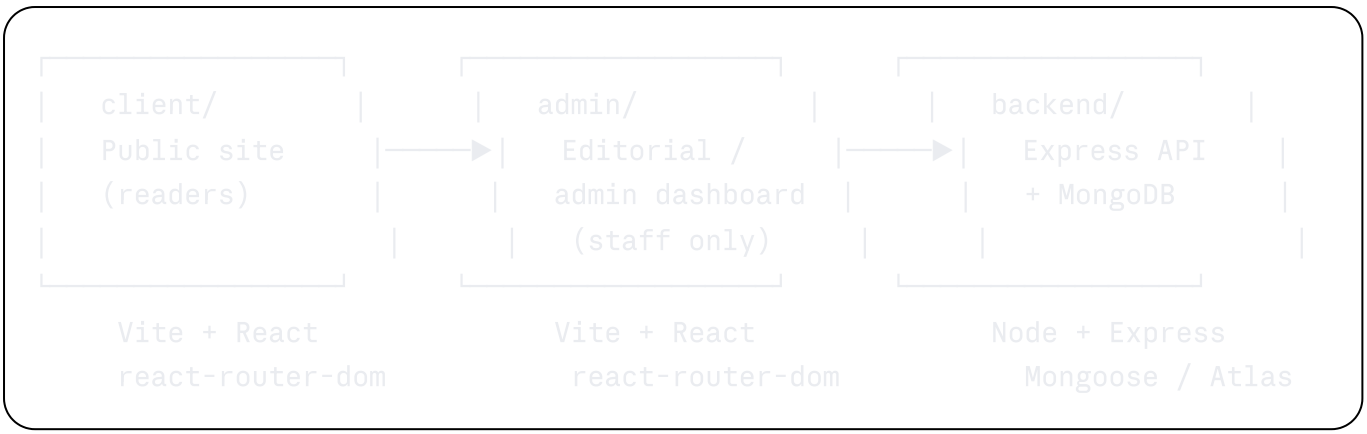
Independent journalism platform covering human rights, minorities, and social justice. A full-stack MERN monorepo made up of three independently deployed applications: a public reader-facing site, an editorial/admin dashboard, and a REST API backend.

> **Note on naming:** the repository/package is called `The_Asr` internally; the product itself is branded **The Orbis Journal**. Both names refer to the same codebase — you'll see `the-asr-*` in deployment URLs and `theorbisjournal.*` in user-facing copy (meta tags, emails, etc.).

Table of contents

Architecture

This is a monorepo containing three separate applications, each deployed as its own Vercel project:



- `client` — the public-facing news site readers land on (`the-asr-1m9a.vercel.app` in the current deployment; production domain configured as `theorbisjournal.in`/`.com` — see Known limitations for a naming inconsistency worth cleaning up).
- `admin` — internal dashboard for editors/admins to manage articles, categories, comments, users, submissions, and newsletter subscribers. Not linked from the public site; accessed directly by staff.
- `backend` — a single Express app (`backend/src/app.js`) exported for both local development (`server.js`) and as a Vercel serverless function. Both `client` and `admin` talk to the same backend/API.

All three apps are deployed independently and configured via their own `vercel.json`.

Tech stack

Layer	Technology
Frontend (client & admin)	React 19, TypeScript, Vite 8, React Router 7, TanStack Query 5, Tailwind CSS 3
Frontend forms/UI (admin only)	React Hook Form, Zod, Recharts, `@hookform/resolvers`
Backend	Node.js, Express 4, Mongoose 8 (MongoDB Atlas)
Auth	JWT (access + refresh tokens), Passport (Google OAuth 2.0), bcryptjs
Media	Cloudinary (image uploads via Multer)
Email	Nodemailer (SMTP)
AI	Google Gemini API (article summarization + Urdu translation, with a DB-level cache)
Security middleware	Helmet, `express-mongo-sanitize`, `hpp`, `express-rate-limit`, custom XSS sanitizer, `sanitize-html`

Layer	Technology
Hosting	Vercel (all three apps, each as a separate project)

Features

Public site (`client`)

- Category, tag, and content-format browsing (news, investigation, opinion, ground report, photo essay, interview, and more — see content types below)
- Full-text article search
- Threaded comments (with moderation)
- Newsletter subscribe/confirm/unsubscribe flow
- Reader submissions (tips/pitches) form
- User accounts: registration, login, Google OAuth, password reset, saved articles, public author profiles
- AI-assisted article summaries and Urdu translation (with right-to-left rendering support)
- "Through the Lens" interactive photo-essay format
- Editor-curated homepage hero (see `setHero` in the admin)

Admin dashboard (`admin`)

- Article editor with draft/review/scheduled/published/archived workflow
- Category management (including nav placement: primary nav vs. "More" dropdown)
- Comment moderation
- User management with role changes and account activation toggles
- Newsletter subscriber list
- Reader submissions inbox with status tracking
- Site-wide stats dashboard (views, users, submissions)

Backend

- Versioned REST API (`/api/v1/...`)
- Scheduled article publishing via a cron-triggered internal endpoint
- Tiered rate limiting (general API traffic, auth, password reset, search, AI endpoints each have their own limits)
- Cookie-based view-count deduplication (prevents trivial view-count inflation)
- Atomic, transaction-backed "set hero article" operation

Roles & permissions

Defined in `backend/src/models/User.js`:

```
subscriber → contributor → editor → admin → superadmin
```

Role checks are enforced per-route via middleware (see `backend/src/middleware/auth.js` and the permission comments in `backend/src/routes/index.js`) — for example, category management is restricted to `admin`/`superadmin`.

Article content types (`backend/src/models/Article.js`): `news`, `investigation`, `opinion`, `ground-report`, `double-lens`, `verified-report`, `photo-essay`, `explainer`, `interview`, `community-voice`, `orbis-original`, `features`

Article statuses: `draft` → `review` → `scheduled` → `published` → `archived`

Project structure

```
The_Asr-main/
├── client/                    # Public site (Vite + React)
│   ├── src/
│   │   ├── pages/             # Route-level pages (Home, Article,
Category, Tag, Format, Search, ...)
│   │   ├── components/
│   │   │   ├── layout/        # Header, Footer, Layout (route Suspense
boundary lives here)
│   │   │   ├── home/          # Homepage-specific sections
│   │   │   ├── article/       # Article rendering (body, photo essay,
comments, etc.)
│   │   │   ├── sidebar/, newsletter/, common/
│   │   ├── context/            # AuthContext
│   │   ├── services/           # api.ts (axios instance + interceptors),
articles.ts, etc.
│   │   ├── hooks/               # useSeoMeta, etc.
│   │   └── index.css            # Tailwind + design tokens (theme colors,
shimmer utility, etc.)
│   └── vercel.json
│
├── admin/                     # Editorial dashboard (Vite + React)
│   └── src/
│       ├── pages/             # Dashboard, ArticleEditor, Categories,
Comments, Users, Submissions, ...
│       └── ...                 # Mirrors client's structural conventions
│
├── backend/                   # Express API
```

```
|   └── src/
|       ├── app.js                # Express app — middleware stack, route
mounting (used by both server.js and Vercel)
|       ├── server.js              # Local dev entrypoint (node
src/server.js)
|       ├── config/                # db.js (Mongo connection caching),
cloudinary.js, passport.js
|       ├── controllers/           # articleController.js +
controllers/index.js (category/comment/newsletter/submission/user)
|       ├── models/                # Article, Category, Comment, Donation,
Newsletter, Submission, User
|       ├── routes/                # auth.js, articles.js, ai.js, index.js
(category/comment/newsletter/submission/user/stats routers)
|       ├── middleware/            # auth.js, errorHandler.js,
rateLimiter.js, sanitize.js, validators.js, multerErrorHandler.js
|       ├── jobs/                  # publishScheduled.js (cron job for
scheduled articles)
|       └── utils/                 # apiResponse.js, email.js, logger.js,
seed.js, tokens.js
|   └── vercel.json
|
└── package-lock.json             # top-level lockfile artifact (no root
package.json / workspaces configured — each app manages its own dependencies
independently)
```

Getting started

Prerequisites

- Node.js 18+ and npm
- A MongoDB Atlas cluster (or local MongoDB with replica-set support — the `setHero` operation uses a transaction, which requires a replica set)
- A Cloudinary account (image uploads)
- A Google Cloud project with OAuth credentials (if you want Google login) and a Gemini API key (if you want AI summaries/translation)
- An SMTP provider (e.g. Gmail app password, SendGrid, etc.) for transactional email

1. Clone and install

Each app manages its own dependencies — install all three separately:

```bash
```

```
git clone <repo-url>
cd The_Asr-main

cd backend && npm install && cd ..
cd client  && npm install && cd ..
cd admin   && npm install && cd ..
```

2. Configure environment variables

Copy the example files and fill them in (see Environment variables below for the full list):

bash

```
cp client/.env.example client/.env
cp admin/.env.example admin/.env
# backend has no .env.example checked in — create backend/.env manually using t
```

3. Seed the database (optional but recommended for local dev)

bash

```
cd backend
npm run seed
```

4. Run all three apps

In three separate terminals:

bash

```
# Terminal 1 — API (defaults to :5000)
cd backend && npm run dev

# Terminal 2 — public site (defaults to :5173, client's .env.example assumes :3
cd client && npm run dev

# Terminal 3 — admin dashboard (defaults to :5174 per admin's .env.example)
cd admin && npm run dev
```

> **Heads up:** `client/.env.example` sets `VITE_ADMIN_URL=http://localhost:5174`, and the backend's CORS allowlist reads `ADMIN_URL`/`CLIENT_URL` from its own `.env`. Make sure the ports you actually run on match what's in `backend/.env`'s `CLIENT_URL`/`ADMIN_URL`, or the API will reject requests with a CORS error.

Environment variables

`backend/.env`

Variable	Purpose
`NODE_ENV`	`development` / `production` — affects logging format, cookie `secure` flag, error verbosity
`PORT`	Local dev server port (unused on Vercel)
`MONGODB_URI`	MongoDB Atlas (or other) connection string
`CLIENT_URL`	Public site origin — used for CORS allowlist
`ADMIN_URL`	Admin dashboard origin — used for CORS allowlist
`JWT_ACCESS_SECRET` / `JWT_ACCESS_EXPIRES`	Access token signing secret + TTL
`JWT_REFRESH_SECRET` / `JWT_REFRESH_EXPIRES`	Refresh token signing secret + TTL
`GOOGLE_CLIENT_ID` / `GOOGLE_CLIENT_SECRET` / `GOOGLE_CALLBACK_URL`	Google OAuth 2.0 credentials (Passport)
`CLOUDINARY_CLOUD_NAME` / `CLOUDINARY_API_KEY` / `CLOUDINARY_API_SECRET`	Media uploads
`GEMINI_API_KEY`	Google Gemini API — AI article summaries + Urdu translation
`SMTP_HOST` / `SMTP_PORT` / `SMTP_USER` / `SMTP_PASS` / `EMAIL_FROM`	Transactional email (password reset, newsletter confirmation, etc.)
`CRON_SECRET`	Bearer token required to call the internal `/api/v1/internal/publish-scheduled` endpoint

`client/.env`

Variable	Purpose	Example
`VITE_API_URL`	Base URL of the backend API	`http://localhost:5000/api/v1`

Variable	Purpose	Example
`VITE_ADMIN_URL`	Used for cross-linking to the admin app where relevant	`http://localhost:5174`
`VITE_SITE_URL`	Canonical site URL, used in SEO meta tags	`https://theorbisjournal.com`

`admin/.env`

Variable	Purpose	Example
`VITE_API_URL`	Base URL of the backend API	`http://localhost:5000/api/v1`
`VITE_CLIENT_URL`	Used for "view live" links back to the public site	`http://localhost:3000`

API overview

All routes are mounted under `/api/v1` (see `backend/src/app.js`):

Prefix	Router	Notes
`/api/v1/auth`	`routes/auth.js`	Register, login, Google OAuth refresh, forgot/reset password
`/api/v1/ai`	`routes/ai.js`	Article summary + translation (Gemini-backed, cached, rate limited separately)
`/api/v1/articles`	`routes/articles.js`	Public listing/search/detail, view increments, admin CRU hero management
`/api/v1/articles/:articleId/comments`	`commentRouter`	Article-scoped comments
`/api/v1/comments/admin`	`adminCommentRouter`	Global comment moderation
`/api/v1/categories`	`categoryRouter`	Public listing/detail, admin CRUD
`/api/v1/newsletter`	`newsletterRouter`	Subscribe/confirm/unsubscri admin subscriber list
`/api/v1/submissions`	`submissionRouter`	Reader tip/pitch submissions, admin review
`/api/v1/users`	`userRouter`	Admin user management, sel profile updates, public author profiles
`/api/v1/admin/stats`	`statsRouter`	Dashboard aggregate stats
`/health`	—	Unauthenticated health check (also used to keep the serverle function warm — see Deployment)
`/api/v1/internal/publish-scheduled`	—	Cron-triggered; requires `Authorization: Bearer <CRON_SECRET>`

Every response follows a consistent envelope (`backend/src/utils/apiResponse.js`):

```json
{ "success": true, "data": { }, "meta": { "page": 1, "limit": 24, "total": 0, "
```

Security

- **Helmet** for standard security headers
- `express-mongo-sanitize` and `hpp` against NoSQL injection / HTTP parameter pollution
- **Custom XSS sanitizer middleware** + `sanitize-html` for rich-text fields
- **CORS allowlist** — only `CLIENT_URL` and `ADMIN_URL` origins are accepted, credentialed requests only
- **JWT access + refresh tokens** — access tokens are short-lived; refresh happens via an `httpOnly` cookie–backed endpoint (see Known limitations re: access-token storage)
- **Tiered rate limiting** (`backend/src/middleware/rateLimiter.js`):
  - General API traffic: 300 requests / 15 min per IP
  - Auth (login/register): stricter dedicated limits
  - Password reset request vs. password reset consumption: separate limiter buckets
  - Search (`?search=` on the articles endpoint): its own limiter, scoped so normal browsing isn't affected
  - AI endpoints (`/summary`, `/translate`): 20 requests / hour per IP
- **View-count dedup** — a short-lived cookie prevents trivial repeated-refresh/script view inflation
- **Atomic hero swap** — `setHero` runs inside a MongoDB transaction so exactly one article is ever featured, even under concurrent admin requests

---

Deployment

Each app is its own Vercel project with its own `vercel.json`:

- `backend/vercel.json` — single serverless function (`src/app.js`), 512 MB / 30s max duration, all paths rewritten into it.
- `client/vercel.json` / `admin/vercel.json` — standard Vite SPA config (build to `dist/`, all paths rewritten to `index.html` for client-side routing).

Cold starts

The backend is a Vercel serverless function, so it cold-starts after periods of inactivity (function spin-up + MongoDB reconnect). To reduce how often users hit this:

- An external scheduler (e.g. cron-job.org) pings `GET /health` every ~5 minutes to keep the function warm. This is **not** configured via Vercel's own Cron Jobs, since Hobby-tier Vercel cron is capped at once/day — an external pinger is used instead.

- The frontend also ships a static, JS-independent loading shell (`client/index.html`) and a branded shimmer skeleton (`.shimmer` in `client/src/index.css`) so that if a cold start does happen, the user sees a styled loading state instead of a blank page.

Environment variables on Vercel

Set the variables from the Environment variables section in each project's Vercel dashboard (Settings → Environment Variables) — `.env` files are for local development only and are not deployed.

Known limitations

Documented here deliberately, rather than left implicit — useful context for anyone (including future you) picking this project back up:

- **Auth token storage is on the roadmap for hardening.** The current session model is functional but not the final target architecture; a cookie-based refresh flow is planned as a follow-up. (Specifics intentionally kept out of this public document — see internal engineering notes.)
- **Cold starts are mitigated, not eliminated.** An external uptime check keeps the backend warm most of the time, and the frontend shows a styled loading state either way, but the backend can still cold-start occasionally (e.g. right after a deploy). Moving to an always-on host would remove this entirely.
- **Domain naming is inconsistent across the codebase**: SEO meta tags reference `theorbisjournal.in`, `client/.env.example` references `theorbisjournal.com`, and the actual current deployment is on a `the-asr-*.vercel.app` Vercel subdomain. Worth consolidating to one canonical domain before/at public launch.
- **No root-level workspace configuration.** `client`, `admin`, and `backend` are three independent npm projects (each with its own `package.json`/lockfile) rather than an npm/pnpm workspace — intentional for now given deployment independence, but means dependency versions (e.g. React, TanStack Query, Vite) must be kept in sync manually across `client` and `admin`.
- **MongoDB transactions require a replica set.** The `setHero` endpoint depends on this. MongoDB Atlas clusters (including the free M0 tier) run as replica sets by default, so this is only a concern if pointing the backend at a standalone local `mongod` instance.

Scripts reference

App	Command	Description
`backend`	`npm run dev`	Start with nodemon (auto-restart on change)

App	Command	Description
`backend`	`npm start`	Start with plain `node` (production)
`backend`	`npm run seed`	Populate the database with seed data
`backend`	`npm run lint`	ESLint over `src/**/*.js`
`client` / `admin`	`npm run dev`	Vite dev server with HMR
`client` / `admin`	`npm run build`	Type-check (`tsc -b`) then production build to `dist/`
`client` / `admin`	`npm run preview`	Preview the production build locally
`client` / `admin`	`npm run lint`	ESLint over the project

License